Git Commands: Practical Reference Guide

Beginner to advanced | Git Bash, GitHub, branching, commits, merge and recovery

A printable learning and troubleshooting companion. Commands are shown for Git Bash. Replace placeholders such as file.txt, dev, origin, and URL with your actual values.

1. First-time setup and repository basics

Command	What it does / when to use
`git --version`	Check installed Git version
`git help -a`	List available Git subcommands
`git help branch`	Open help for a subcommand
`git config --global user.name "Your Name"`	Set commit author name
`git config --global user.email "you@example.com"`	Set commit author email
`git config --list`	View Git configuration
`git init`	Create a repository in the current folder
`git clone URL`	Clone an existing repository
`git status`	Show staged, unstaged and untracked changes
`git status -s`	Short status output
`git rev-parse --show-toplevel`	Print repository root folder

2. Add, commit, inspect

Command	What it does / when to use
`git add file.txt`	Stage one file
`git add .`	Stage changes in current folder and descendants
`git add -A`	Stage all changes in the repository
`git add -p`	Stage selected change hunks interactively
`git restore --staged file.txt`	Unstage without discarding file changes
`git restore --staged .`	Unstage changes under current folder
`git commit -m "message"`	Commit staged changes on current branch
`git commit --amend`	Update latest commit; beware if already pushed
`git log`	Show commit history
`git log --oneline`	Compact commit history
`git log --graph --oneline --all`	Visualize branch and commit graph
`git show HEAD`	Inspect latest commit
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes
`git diff main..dev`	Compare branch tips
`git ls-files`	Show tracked files

3. Branches: create, list, switch and delete

Command	What it does / when to use
`git branch`	List local branches with existing commits
`git branch -a`	List local and remote-tracking branches
`git branch -r`	List remote-tracking branches
`git branch -vv`	Show tracking and latest commits
`git branch --show-current`	Show current branch name
`git switch -c dev`	Create and switch to dev
`git branch dev`	Create dev without switching
`git switch dev`	Switch to existing dev branch
`git switch main`	Switch to main
`git branch -m new-name`	Rename current branch
`git branch -d dev`	Safely delete merged local branch
`git branch -D dev`	Force-delete local branch; use carefully
`git merge dev`	Merge dev into currently checked-out branch
`git merge --abort`	Abort a conflicted merge

Why git branch -a might show nothing: In a newly initialized repository with no commits, main can appear in the shell prompt but the branch reference does not yet exist. Run `git status` and `git log --oneline`. Create a first commit containing a file, then list branches again.

4. GitHub remotes: fetch, pull and push

Command	What it does / when to use
`git remote -v`	List configured remote URLs
`git remote add origin URL`	Connect a local repository to a remote
`git remote set-url origin URL`	Change an existing remote URL
`git remote remove origin`	Remove remote configuration
`git fetch origin`	Download remote branches and commits without merging
`git fetch --all --prune`	Refresh all remotes and prune stale remote refs
`git ls-remote --heads origin`	See branches directly on remote
`git pull`	Fetch and integrate remote changes
`git pull --rebase`	Fetch and rebase local commits
`git push -u origin main`	Push main and set its upstream
`git push -u origin dev`	Push dev and set its upstream
`git push`	Push current branch to configured upstream
`git push origin --delete dev`	Delete dev on the remote
`git branch -M main`	Rename current branch to main forcibly

5. Undo, recover and stash safely

Command	What it does / when to use
`git restore file.txt`	Discard unstaged changes in tracked file (destructive)
`git reset`	Unstage changes, retain working-tree edits
`git reset --soft HEAD~1`	Undo one commit, keep changes staged
`git reset --mixed HEAD~1`	Undo one commit, keep changes unstaged
`git reset --hard HEAD~1`	Discard a commit and working-tree changes (destructive)
`git revert HEAD`	Revert last commit with a new commit; useful for shared history
`git reflog`	Find prior HEAD positions for recovery
`git stash`	Temporarily save changes to tracked files
`git stash -u`	Stash tracked and untracked changes
`git stash list`	List saved stashes
`git stash pop`	Restore and remove stash if applied successfully
`git stash apply`	Restore stash without dropping it
`git stash show -p`	Show stashed patch
`git stash drop`	Remove one saved stash
`git clean -n`	Preview untracked-file removal
`git clean -fd`	Delete untracked files/directories (destructive)

Safety: Always inspect `git status` and consider a backup before running `reset --hard`, `clean -fd`, `branch -D`, or restoring files.

6. Merge, rebase, cherry-pick and tags

Command	What it does / when to use
`git rebase main`	Replay current branch commits on main
`git rebase -i HEAD~3`	Interactively edit last three commits
`git rebase --continue`	Continue rebase after resolving conflicts
`git rebase --abort`	Cancel in-progress rebase
`git cherry-pick COMMIT_ID`	Apply one commit to current branch
`git tag`	List local tags
`git tag -a v1.0.0 -m "Release"`	Create annotated version tag
`git push origin v1.0.0`	Publish one tag
`git push origin --tags`	Publish all local tags
`git bisect start`	Start binary search for a problematic commit
`git blame file.txt`	Show line-level commit attribution
`git grep "pattern"`	Search tracked content

7. Practical workflow: initial commit and feature branch

```
mkdir git-practice
cd git-practice
git init
git branch -M main
echo "# Git Practice" > README.md
git add README.md
git commit -m "Initial commit"
git branch -a
git switch -c dev
echo "Learning branches" >> README.md
git add README.md
git commit -m "Update README on dev"
git switch main
git merge dev
git log --graph --oneline --all
```

To publish: Create an empty GitHub repository first, then use the two commands below (replace the URL).

```
git remote add origin https://github.com/USERNAME/git-practice.git
git push -u origin main
```

8. Practical workflow: pull request from the command line

A GitHub pull request is not a local Git object. With Git alone you can fetch and merge its changes locally; opening/merging the PR on GitHub normally uses the GitHub website or GitHub CLI (gh).

```
git fetch origin pull/123/head:pr-123
git switch main
git merge pr-123
git push origin main
```

Replace 123 with the PR number. The fetch syntax shown is for GitHub; ensure you review and test the branch before merging. Alternative using GitHub CLI: `gh pr checkout 123` and `gh pr merge 123 --merge`.

9. Troubleshooting quick checks

Command	What it does / when to use
`git status`	Confirm repository, current branch and pending changes
`git log --oneline -5`	See whether first commit exists
`git branch -a`	Check refs available locally
`git remote -v`	Confirm origin is configured
`git fetch --all --prune`	Update remote-tracking branches
`git ls-remote --heads origin`	Check branches actually present on GitHub
`git config --get user.email`	Check configured author email
`git diff --check`	Check whitespace problems in patches
`git check-ignore -v file.txt`	Explain why a path is ignored
`git fsck`	Run repository integrity check

10. Quick mental model

Working directory (edit) -> Staging area (`git add`) -> Local repository (`git commit`) -> GitHub (`git push`). Use `git fetch` to retrieve remote updates and `git pull` to retrieve plus integrate.

DevOps security tip: Add .terraform/, *.tfstate, *.tfstate.*, and sensitive *.tfvars to .gitignore before staging Terraform projects. Never commit passwords, tokens or private keys.

Reference: official Git documentation at https://git-scm.com/docs and GitHub documentation at https://docs.github.com/